

WEIGHT SPACE LEARNING WITH MIXTURE OF EXPERTS (MoE)

Lorenzo Sciarretta

AI-ML Lab

University of St. Gallen (HSG)

St. Gallen, Switzerland

lorenzo.sciarretta@unisg.ch

Zakaria Omarar

AI-ML Lab

University of St. Gallen (HSG)

St. Gallen, Switzerland

zakaria.omarar@student.unisg.ch

ABSTRACT

Weight-space learning is an emerging field in deep learning that aims to learn representations of *trained* neural networks directly from their parameters, with the goal of better understanding model behavior and enabling downstream discriminative and generative tasks. Prior work has shown that Transformer-based autoencoders can learn “hyper-representations” from collections of networks and leverage these representations for tasks such as model classification and weight generation. However, existing approaches struggle to scale beyond homogeneous datasets, and their performance can degrade when trained on more diverse model families or substantially larger networks.

In this work, we study how to learn robust and transferable weight-space representations from heterogeneous collections of neural networks. We introduce a Mixture-of-Experts (MoE) extension of a Transformer-based weight autoencoder to better handle dataset diversity. Experiments show that MoE improves accuracy of downstream tasks on mixed datasets and consistently outperforms the baseline across our evaluation tasks.

1 INTRODUCTION

Deep neural weights span a vast *weights space*: for a given architecture and training objective, every set of learned parameters defines a point in a high-dimensional space. Exploring this space can reveal patterns about how networks generalize and how architectures may be generated. Prior works have investigated weight distributions and structural statistics to predict model quality or generalization gaps, for example by examining margin distributions (Yak et al., 2019), (Jiang et al., 2019), graph-topology features (Corneanu et al., 2020), or singular-value spectra. Beyond discriminative tasks, a parallel thread of research uses hypernetworks (Ha et al., 2016) to generate new model weights for neural architecture search, compression, or meta-learning. These generative approaches train hyper-networks or hyper-transformers (Zhmoginov et al., 2022) on a fixed dataset to emit weights for target architectures.

To avoid relying on the original training data, hyper-representation (Schürholt et al., 2022a) methods learn latent codes directly from weight space. Early hyper-representation autoencoders compress each trained model to a single vector and use it for downstream prediction or generation. However, these methods are limited in two critical ways: they require embedding the entire flattened weight vector, which becomes infeasible for large networks, and they can only embed models with architectures identical to those seen during training. As a result, separate hyper-representations must be trained for different tasks and architectures, hampering scalability and versatility.

To address the scalability challenge, the *Sequential Autoencoder for Neural Embeddings (SANE)*, (Schürholt et al., 2024), introduced a novel approach by treating neural weights not as a monolithic vector, but as a sequence of discrete “tokens,” analogous to sentences in Natural Language Processing. By processing these tokens sequentially, SANE utilizes the Transformer architecture (Vaswani et al., 2023) to handle models of arbitrary depth. However, the standard SANE architecture relies on a dense Transformer backbone, where every parameter in the Feed-Forward Networks (FFNs) is activated for every incoming weight token. This creates a “generalist” dynamic that struggles with

the heterogeneity of neural architectures. In this setup, a single dense FFN must simultaneously learn to process the weights of NN that may be very different in terms of architecture or training data. In heterogeneous model zoos, this lack of specialization leads to destructive interference, where the model’s capacity is saturated not by the volume of data, but by the diversity of the distributions it must model.

To overcome the limitations of dense backbones of SANE framework, we propose the integration of Mixture of Experts (MoE). Inspired by recent advancements in scaling language models (Shazeer et al., 2017; Fedus et al., 2022b), we introduce SANE-MoE. This architecture employs conditional computation, where only a sparse subset of parameters, specific ”experts”, are activated for any given input token.

This extension is motivated by the inherent heterogeneity of weight space: different layers, architectures, datasets, and functional components exhibit distinct patterns that a single shared encoder must otherwise model jointly. By allowing experts to specialize on different regions of this space, SANE-MoE reduces interference between unrelated weight types and increases the model’s effective capacity without increasing computational cost at inference time. As a result, the encoder can learn sharper and more expressive hyper-representations.

In this paper, we present the SANE-MoE architecture, outlining the key components that enable Mixture-of-Experts techniques to be applied effectively to weight-space learning. We conduct an extensive empirical evaluation to assess the performance and benefits of this approach.

2 RELATED WORK

Weight-space learning and hyper-representations. A growing body of work studies the geometry and statistics of neural network weight spaces as a means to characterize trained solutions or predict generalization, for example through margin- or sharpness-based measures (Yak et al., 2019; Jiang et al., 2019) or structural and topological descriptors (Corneanu et al., 2020). Moving beyond scalar diagnostics, *hyper-representation* methods aim to learn compact latent representations of entire trained networks directly from their parameters, enabling downstream discriminative and generative tasks without access to the original training data (Schürholt et al., 2022a). These works highlight both the promise of weight-space representations and key challenges, including scalability to larger models and robustness to diverse collections of trained networks (often referred to as “Model Zoos”, (Schürholt et al., 2022b)).

Hypernetworks and weight generation. A related line of research explores hypernetworks that generate the parameters of a target model conditioned on task, architecture, or context (Ha et al., 2016; Zhmoginov et al., 2022). Such approaches are commonly applied in meta-learning, model compression, and architecture search, and typically assume a fixed or explicitly parameterized family of architectures. In contrast, weight-space learning and hyper-representation methods operate on *already trained* checkpoints and aim to infer latent codes that capture functional properties of the solution manifold, optionally enabling generation or interpolation directly in weight space.

Sequential modeling of weights. Early hyper-representation autoencoders often flatten model parameters into a single vector, which limits scalability and assumes a fixed architecture. SANE addresses these limitations by modeling parameters as a sequence of weight tokens and applying a Transformer-based autoencoder, enabling scalable encoding of deep networks and support for variable-depth models via tokenization and positional or layer encodings (Schürholt et al., 2024). SANE provides the most direct baseline for our work; however, its dense Transformer backbone acts as a single generalist over diverse weight-token distributions, which becomes increasingly challenging when training on heterogeneous model zoos.

Mixture-of-Experts and conditional computation. Mixture-of-Experts (MoE) architectures increase model capacity through conditional computation, routing each token to a sparse subset of expert networks (Shazeer et al., 2017). Large-scale MoE Transformers, such as Switch Transformers (Fedus et al., 2022b), demonstrate that sparse routing can improve efficiency and performance, but also introduce challenges related to routing stability, expert imbalance, and capacity control. While MoE has been primarily studied in language and vision domains, recent work has emphasized the

importance of *structured routing signals* that align expert specialization with meaningful token attributes.

Structured routing and specialization. Several recent approaches investigate routing mechanisms that promote expert specialization and reduce interference, including context-aware routing, auxiliary balancing objectives, and hierarchical expert structures (Zhao et al., 2019; Zhou et al., 2022; Fedus et al., 2022a). In particular, MAPEX (Hanna et al., 2025) introduces modality-conditioned routing in a Mixture-of-Experts foundation model for remote sensing, demonstrating that conditioning routing decisions on token-level semantic structure (i.e., data modality) elicits specialized experts and improves downstream performance. Although MAPEX operates in a different domain, it provides strong evidence that explicitly encoding token structure into the routing mechanism is critical for effective expert specialization. In weight-space learning, tokens are similarly structured: weight blocks are not exchangeable, but are organized by layer type, layer index, and spatial coordinates (e.g., convolutional kernels). Our approach leverages this inductive bias by conditioning routing on weight-token content together with positional and layer-index information, encouraging experts to specialize to distinct regions of weight space.

Summary of novelty. To the best of our knowledge, prior work has not combined MoE-style conditional computation with sequential weight-space autoencoders to learn hyper-representations that scale to heterogeneous model zoos. Our work makes three main contributions: (i) we introduce **SANE-MoE**, a Mixture-of-Experts extension of the SANE Transformer autoencoder for sequential weight tokens; (ii) we adapt MoE **routing, capacity control, and load-balancing objectives** to the weight-token domain, including **routing strategies** that condition on token content and structural metadata (e.g., position and layer index) to encourage specialization; and (iii) we provide an extensive empirical evaluation on mixed CNN datasets, showing that structured expert specialization improves reconstruction (R^2_{val}) and downstream accuracy in both reconstruction and generative tasks compared to the dense SANE-standard baseline.

3 METHOD

3.1 BACKGROUND: SEQUENTIAL AUTOENCODER FOR NEURAL EMBEDDINGS (SANE)

SANE (Schürholt et al., 2024) is a Transformer-based autoencoder that learns hyper-representations directly from trained network parameters by tokenizing weights into a sequence. Given a checkpoint, parameters are grouped into layer-wise blocks, standardized, and serialized into a fixed-length token sequence. An encoder Transformer maps the token sequence to latent embeddings, and a symmetric decoder reconstructs weight tokens, which are then de-tokenized back into a full parameter set. This sequential design scales to larger networks by operating on local token windows rather than flattening the entire parameter vector, while preserving structure via positional and layer encodings.

3.2 BASE ARCHITECTURE AND MIXTURE-OF-EXPERTS DESIGN

Our backbone builds on the SANE sequential autoencoder (Schürholt et al., 2024). Following SANE, both the encoder and decoder are stacks of Transformer encoder blocks with positional embeddings. To increase capacity without linearly increasing computation, inspired by recent advances in large language models, we replace the Feed-Forward Network (FFN) sub-layer in selected Transformer blocks with a Mixture-of-Experts (MoE) layer (Shazeer et al., 2017). Throughout the paper, we refer to the original architecture of (Schürholt et al., 2024) as **SANE-standard** and to our extension as **SANE-MoE**.

In the baseline SANE-standard, each FFN consists of a position-wise MLP (linear $\rightarrow$ GELU $\rightarrow$ dropout $\rightarrow$ linear). We keep this as the architecture of each expert, but we allow multiple such experts and a router to select among them, as shown in Figure 1. At a high level, an MoE layer consists of a set of N expert networks $\{E_1, \dots, E_N\}$ and a gating network (router) G . For an input token representation $h \in \mathbb{R}^d$, the output of the MoE layer is:

$$y = \sum_{i=1}^N G(h)_i E_i(h), \quad (1)$$

where $G(h)_i$ is the normalized routing weight assigned to the i -th expert. In our implementation, each expert E_i is a two-layer feed-forward network with GELU activation and dropout: $x \mapsto W_2 \sigma(W_1 x)$.

Moreover, to encourage uniform usage of the experts, we compute a balance loss that penalizes deviations from uniform importance (average gate probability across tokens) and load (fraction of tokens assigned to each expert), as proposed by (Shazeer et al., 2017).

We have implemented four routing strategies: (1) **Top- k routing**, where only the k highest-scoring experts are activated per token and re-normalized; (2) **soft (dense) routing**, where the router produces a softmax distribution over all experts; (3) **context-aware position-dependent routing**, where routing is conditioned on token content and spatial context (via positional features and a layer-index embedding) and can include a learnable spatial bias; and (4) **hierarchical spatial routing**, where routing is decomposed into a coarse region assignment followed by region-conditioned expert selection. We describe each routing strategy in detail in the following subsections.

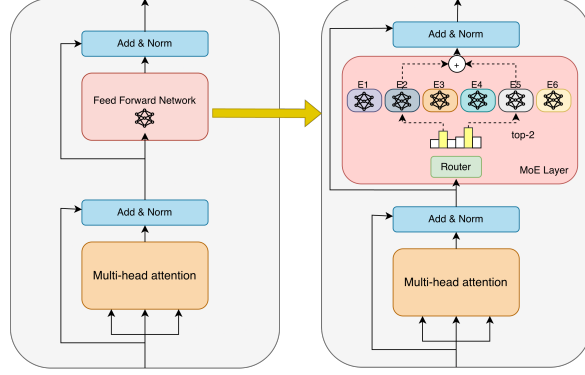


Figure 1: **SANE-MoE encoder block**. The standard FFN is replaced by a Mixture-of-Experts module (router + N experts) in selected layers. In this figure the routing strategy is Top- k , $k = 2$.

3.3 MOE-BASELINE OPTIMIZATION: EXPERT COUNT AND SPARSITY

To establish a strong SANE-MoE baseline, we perform a systematic hyperparameter search over the total number of experts N and the number k of active experts per token. Under **Top- k routing**, the router computes logits $l = W_g h \in \mathbb{R}^N$ and gates $\pi = \text{softmax}(l)$, then activates only the k experts with the largest gate values and re-normalizes the retained weights:

$$G(h) = \text{Normalize}(\text{Top-}k(\pi, k)), \quad (2)$$

where $\text{Top-}k(\cdot, k)$ sets all but the largest k elements to zero; the special case $k = 1$ corresponds to hard (Top-1) routing (Fedus et al., 2022b). Increasing N increases model capacity but also memory footprint; decreasing k enforces sparsity and reduces per-token computation. We fix all other components (Transformer depth, latent dimension, positional encoding, optimizer, learning-rate schedule, dropout, weight decay) to isolate the effect of MoE configuration. For each (N, k) combination, we train the autoencoder with the same protocol and monitor the coefficient of determination R_{val}^2 on the validation set.

3.4 COMPARISON WITH DENSE ROUTING REGIME

To disentangle the effect of conditional computation from that of increased model capacity, we also consider a dense routing strategy. In the dense routing regime, all N experts contribute to every token, corresponding to the case $k = N$. The router therefore omits the Top- k truncation and computes gating weights as $G(h) = \text{softmax}(W_g h)$, so that the output representation y aggregates the outputs of all experts. This setting reduces to the original FFN when $N = 1$, and serves as the dense MoE reference when $N > 1$. While this setting incurs higher computational cost, which scales linearly with N rather than k , it provides a useful reference for evaluating the trade-off between sparsity and representational capacity.

3.5 CONTEXT-AWARE ROUTING

Standard Top- k routing selects experts based only on token content and treats tokens independently, ignoring structural information that is naturally present in neural weight spaces. To incorporate such structure, we introduce a **context-aware routing** mechanism that conditions expert selection on both content and spatial context.

The router takes three inputs: (i) the content embedding h , (ii) a positional embedding p encoding spatial information such as kernel row, column, and channel indices (of the original CNN), and (iii) a learnable embedding representing the MoE layer index l_{idx} . These embeddings are concatenated and passed through an MLP to produce content-based routing logits l_{content} .

To introduce spatial inductive bias, we add a spatial term to the logits:

$$l = l_{\text{content}} + \tau p B_{\text{spatial}}^{\top},$$

where B_{spatial} is a matrix encoding expert-specific spatial preferences and τ controls the strength of the spatial bias. When $\tau = 0$, routing depends only on content; increasing τ encourages experts to specialize in specific geometric regions.

The router supports both soft routing (Top- k with $k > 1$) and hard routing (Top-1).

3.6 HIERARCHICAL SPATIAL ROUTING

To further explore more detailed routing strategies, we have implemented **Hierarchical Position-Based Routing**. The hierarchical router decomposes expert selection into two stages. First, each token is softly assigned to a spatial region based on its positional encoding, using a region classifier. Second, conditioned on the selected region, a content-based router selects experts only within the corresponding subset of experts assigned to that region. The final expert gates are obtained by weighting region-specific expert scores with the region assignment probabilities and normalizing across all experts. This design enforces spatial specialization while retaining content adaptivity, and allows us to study whether structured expert partitioning improves reconstruction quality.

4 EXPERIMENTS

In this section we evaluate our SANE-MoE and compare it to the SANE-standard baseline. We first describe the general evaluation protocol and selection criterion, then present the results of three stages of experiments: a baseline grid search over the number of experts and the Top- k parameter, a study of position-aware routing, and an exploration of hierarchical position routing. After each stage we analyse the routing behaviour to understand whether experts specialize to particular model zoos or spatial regions. We conclude with a summary of validation and downstream results.

4.1 EXPERIMENTAL SETUP

Both SANE-standard (the baseline) and our SANE-MoE variants are trained with the same optimization protocol: the same optimizer, learning-rate schedule, batch size, and number of training epochs (Section F). For each hyperparameter configuration we monitor the coefficient of determination R_{val}^2 on a held-out validation set of models. For each hyperparameter configuration, we train the autoencoder and select the model checkpoint with the best validation reconstruction score R_{val}^2 . Unless stated otherwise, we report the *best* R_{val}^2 .

In Section 4.5 we confirm that improvements in R_{val}^2 translate into stronger performance on the selected downstream tasks.

We consider two experimental settings:

- **Single-Zoo.** The autoencoder is trained and evaluated on models from a single Zoo dataset (e.g. CIFAR-10 Zoo or STL-10 Zoo) (Schürholt et al., 2022b). These results are reported in Section B in the appendix; we focus our main analysis on the more challenging Multi-Zoo setting.

- **Multi-Zoo.** The autoencoder is trained and evaluated on a combined corpus of models from two Zoos datasets (CIFAR-10 Zoo **and** STL-10 Zoo). This heterogeneous setting benefits most from conditional computation and specialization because the distribution of network weights is more diverse.

Each dataset provides a collection of independently trained CNN checkpoints (same base architecture and training recipe, different random seeds and training trajectories) as in Schürholt et al. (2022b). In Multi-Zoo, we merge two single datasets, increasing diversity in the weight distribution due to differences in data and learned features.

All hyperparameter searches described below are conducted on the **Multi-Zoo** setup unless otherwise noted.

4.2 STAGE 1: TOP- k ROUTER GRID SEARCH

We begin by investigating the effect of expert capacity and routing sparsity. Using the standard Top- k router, we perform a full grid search over the number of experts $N \in \{2, 4, 6, 8\}$ and the number of active experts per token $k \in \{1, 2, 3\}$. Larger N increases model capacity but also computation; smaller k enforces stronger sparsity and reduces computation. We choose $k \leq 3$ to keep the computational benefit of the MoE architecture: although increasing the number of experts N increases the total number of model parameters, the dominant per-token expert computation scales with k rather than N . As a result, MoE enables higher model capacity at approximately constant computational cost compared to a dense FFN, while still allowing multiple experts to contribute to each token. For each (N, k) combination we train the autoencoder and record the maximum R_{val}^2 on the Multi-Zoo. See results in Section A of the appendix.

Analysis: Top- k Router. Across the grid, the configuration with $N = 4$ experts and $k = 2$ active experts achieves the best R_{val}^2 while remaining sparse; we therefore adopt this setting for all subsequent routing experiments. From a further analysis, we observed a consistent depth-dependent pattern: in early layers, expert selection is primarily structured by token position, reflecting architectural organization of the serialized weights. At intermediate layers, routing begins to differ across datasets, indicating the emergence of dataset-conditioned expert preferences. This results in partial expert-dataset specialization, where some experts are biased toward a specific dataset while others remain shared. Further routing diagnostics (depth-wise routing heatmaps, per-layer expert usage by zoo, positional entropy/sharpness, and expert ablation) are deferred to Section C in the appendix.

4.3 STAGE 2: POSITION-AWARE ROUTING

Motivated by the Stage 1 observations, we extend the router with explicit positional context. Specifically, we form the routing input by concatenating the token embedding, a projected positional encoding, a learnable layer-index embedding, and we add a learned spatial bias scaled by a temperature parameter τ (see detailed explanation in Section 3.5). Keeping the MoE configuration fixed ($N = 4$ experts, Top- $k = 2$), we perform a grid search over $\tau \in \{0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875, 1\}$. Larger τ increases the influence of the spatial information and encourages position-based expert specialization, whereas smaller τ reduces this effect and makes routing more content-driven. Table 1 reports validation results. The best value is $\tau = 1$ (the strongest positional bias tested), suggesting that stronger structured/positional routing is beneficial in our setting and motivating further exploration of more expressive routers (e.g., hierarchical routing).

Table 1: Validation R_{val}^2 for the positional temperature grid search. Each cell will be replaced with the measured R_{val}^2 . Bold indicates the best value.

τ	0.125	0.25	0.375	0.5	0.625	0.75	0.875	1
R_{val}^2	0.8993	0.8975	0.8995	0.8982	0.9003	0.8996	0.8996	0.9016

4.4 STAGE 3: HIERARCHICAL SPATIAL ROUTING

The results of the positional grid search in Table 1 suggest that strong spatial bias improves reconstruction. We therefore propose a *hierarchical spatial routing* strategy: a coarse classifier assigns tokens to one of R spatial regions, and within each region a content-based router selects among a subset of experts. Based on the architecture of our model zoos, which have four convolutional layers, we consider $R = 4$ regions. For computational reasons we test either one expert per region (1 expert/region) or two experts per region (2 expert/region). All experiments fix $N = 4$ experts and $k = 2$; only the region assignment varies. Table 2 report the validation results. The best configuration uses one expert per region ($R = 4$), which yields the highest R_{val}^2 and is retained for analysis.

Table 2: Validation R^2 for the hierarchical routing search. Each cell will be replaced with the measured R_{val}^2 . The notation “1 ex/region” means one expert per region and “2 ex/region” means two experts per region.

Regions R	1 ex/region	2 ex/region
4	0.921	0.8378

4.5 SUMMARY AND DOWNSTREAM EVALUATION

To assess whether improvements in reconstruction quality translate into *functional* gains, we evaluate our models on the **Downstream Generative Task** introduced by Schürholt et al. (2024). In brief, the downstream tasks measures (i) how well an autoencoder can reconstruct a checkpoint while preserving its task performance (*accuracy retention of reconstruction*), and (ii) whether the learned latent space can generate novel models with non-trivial downstream accuracy. A detailed description of the task and protocol is provided Section D in the appendix.

In Table 3, we compare the **SANE-standard** baseline with our **SANE-MoE** variants employing three routing strategies: (i) standard Top- k routing (four experts, $k = 2$), (ii) position-aware routing (four experts, $k = 2$ and $\tau = 1$), and (iii) hierarchical routing (four experts, $k = 2$, two experts per region). All models are trained for 5000 epochs under the **Multi-Zoo** setting, obtained by combining two single-zoos, CIFAR-10 small and STL-10 small. We report two complementary evaluation metrics.

First, we assess reconstruction quality on the Multi-Zoo using a held-out validation set and select the best-performing configuration based on the validation coefficient of determination, R_{val}^2 . Second, we evaluate the **downstream reconstruction task** by decoding reconstructed checkpoints and measuring their test accuracy on each underlying image dataset (**CIFAR-10** (Krizhevsky) and **STL-10** (Coates et al.)). Across all configurations, MoE-based backbones consistently outperform the dense SANE-standard baseline. Moreover, increasingly structured routing mechanisms yield further improvements downstream accuracy.

Because higher R_{val}^2 typically coincides with better downstream accuracy in our runs, we run the downstream evaluation only for the best configuration within each router family (selected by R_{val}^2) (see Section 4.2, Section 4.3, and Section 4.4).

Beyond reconstruction, we assess whether improved hyper-representations translate into stronger generative behavior. Specifically, the generative downstream setting samples latent codes from the learned embedding space (via KDE over high-performing anchor models) and decodes them into *novel* checkpoints, which are then evaluated on the corresponding images test sets (see Appendix, Section D for a detailed explanation). Table 4 reports the mean test accuracy of these generated models for CIFAR-10 small and STL-10 small Zoos. This accuracy metric captures whether the latent space learned by the autoencoder creates weight space in a way that supports sampling of non-trivial, functional networks rather than merely reconstructing existing checkpoints.

Consistent with the reconstruction setting, Table 4 shows that all SANE-MoE variants outperform the SANE-standard baseline in the **generative task**. To better understand the results, we display in Figure 2 the probability density of the test accuracy over ≈ 150 models, in red the original CNN models, in yellow the reconstructed one and in blue the sampled one, in Figure 2a using SANE-standard to reconstruct and generate models, and in Figure 2b using SANE-MoE with position-aware routing to reconstruct and generate models. The distributions in the figures suggest that SANE-MoE

Table 3: Validation performance and downstream accuracy of reconstructed models on **Multi-Zoo** (CIFAR-10 small + STL-10 small). We report the best validation R^2_{val} and the downstream test accuracy of reconstructed models (mean over 150 reconstructions) on CIFAR-10 and STL-10 (Image datasets). The last row reports the original CNN test accuracy for reference. MoE-based SANE variants yield reconstructed models whose accuracies are closer to the originals than SANE-standard. The row marked *Best* denotes the best configuration within each routing strategy.

Model	R^2_{val}	CIFAR-10 acc. (mean)	STL-10 acc. (mean)
SANE-standard (baseline)	0.810	38.92%	29.68%
Top- k router (best)	0.893	42.48%	31.23%
Position-aware router (best)	0.902	42.52%	31.14%
Hierarchical router (best)	0.921	41.43%	31.35%
Original CNN models	—	45.81%	34.93%

Table 4: Downstream **generative** results on **Multi-Zoo** (CIFAR-10 small + STL-10 small). We report the mean (over 150 models sampled) test accuracy of *generated* checkpoints produced by sampling in the learned latent space and decoding back to weights. In the last row we report also the original CNN model accuracy for reference. The MoE versions of SANE achieve a closer accuracy to the original one than SANE-standard.

Model	CIFAR-10 gen. acc. (mean $\pm$ std) [%]	STL-10 gen. acc. (mean $\pm$ std) [%]
SANE-standard (baseline)	38.74 $\pm$ 4.16	28.90 $\pm$ 5.43
Top- k router (best)	43.88 $\pm$ 3.01	32.13 $\pm$ 4.38
Position-aware router (best)	43.07 $\pm$ 3.20	30.44 $\pm$ 5.31
Hierarchical router (best)	42.41 $\pm$ 2.19	32.88 $\pm$ 4.01
Original CNN models	45.81	34.93

with positional-aware routing reconstruct and generate models that are much closer to the original CNNs accuracy.

5 CONCLUSION

In this paper, we proposed SANE-MoE, an enhanced version of the SANE model (Schürholt et al., 2024) for learning representations in the weight space. Our approach integrates a Mixture-of-Experts architecture into the SANE backbone to better handle heterogeneous collections of trained CNNs, commonly referred to as Model Zoos, (Schürholt et al., 2022b). We explored multiple routing strategies and demonstrated that SANE-MoE achieves improved reconstruction and generative performance, yielding downstream accuracies closer to those of the original CNNs than the SANE-standard baseline.

A key takeaway is that sparsity is central to making MoE practical in this setting: while Top- k *dense* routing (k = all experts) can improve performance, Top- k *sparse* routing with small k preserves the computational benefit of MoE and offers a more meaningful accuracy/efficiency trade-off.

Future work includes strengthening the underlying Transformer backbone (e.g., SwiGLU (Shazeer, 2020), improved depth/width tuning) and exploring more adaptive routing mechanisms such as dynamic routing (Guo et al., 2025) to further improve specialization without sacrificing efficiency. Finally, our main empirical evaluation focuses on CNN-based Multi-Zoo settings (CIFAR-10 small + STL-10 small); extending SANE-MoE to larger and more modern architectures (e.g., ResNets (He et al., 2016)) and to broader, more heterogeneous Zoos mixtures ((Falk et al., 2025)) is an important direction for future work.

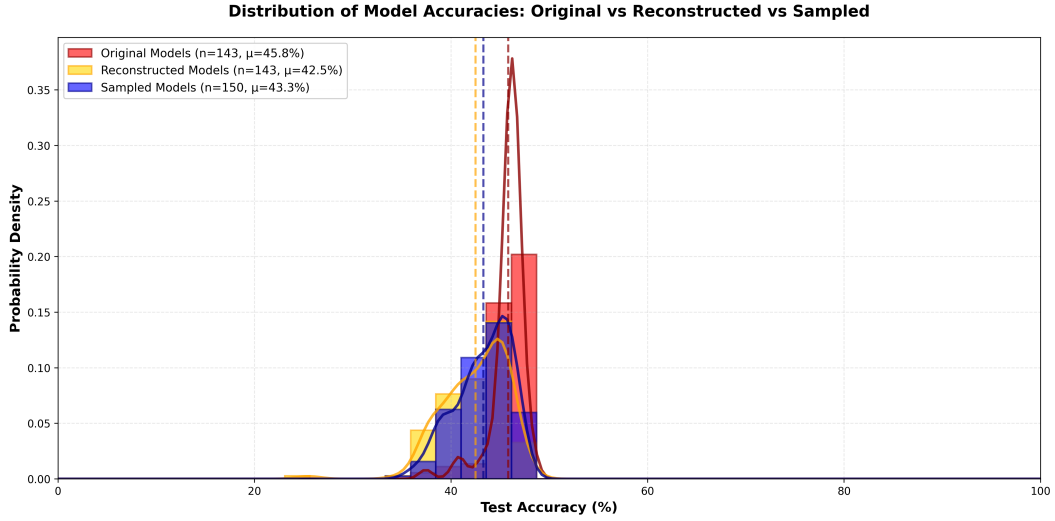
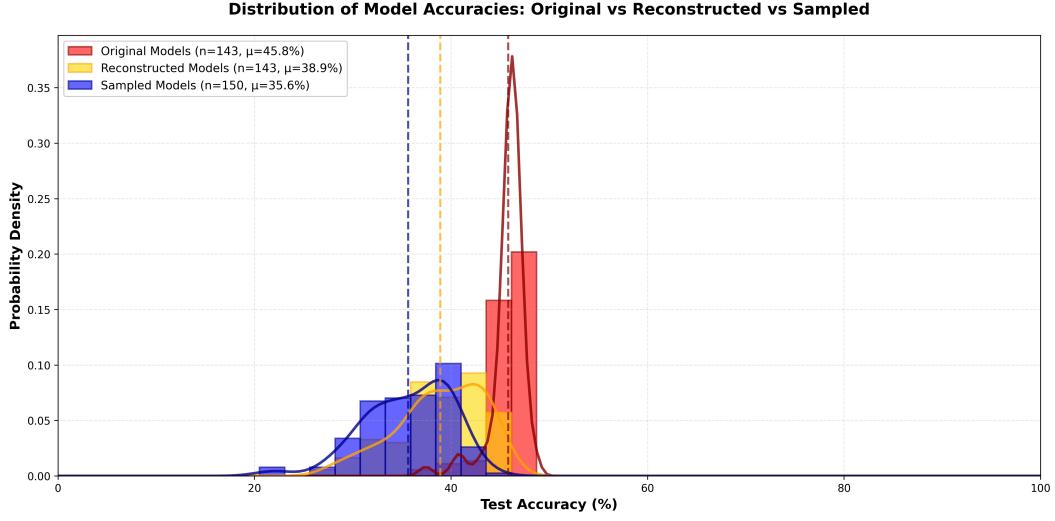


Figure 2: Probability density of test accuracy over approximately 150 models on CIFAR-10 small Zoo. Original CNN models are shown in red, reconstructed models in yellow, and sampled models in blue. (a) Results obtained using SANE-standard for reconstruction and generation, trained on Multi-Zoo (CIFAR-10 + STL-10). (b) Results obtained using SANE-MoE with position-aware routing, trained on Multi-Zoo (CIFAR-10 + STL-10).

REFERENCES

- Adam Coates, Honglak Lee, and Andrew Y Ng. An Analysis of Single-Layer Networks in Unsupervised Feature Learning.
- Ciprian Corneanu, Meysam Madadi, Sergio Escalera, and Aleix Martinez. Computing the Testing Error without a Testing Set, May 2020. URL <http://arxiv.org/abs/2005.00450>. arXiv:2005.00450 [cs].
- Damian Falk, Konstantin Schürholt, Konstantinos Tzevelekakis, Léo Meynent, and Damian Borth. Learning Model Representations Using Publicly Available Model Hubs, October 2025. URL <http://arxiv.org/abs/2510.02096>. arXiv:2510.02096 [cs].

- William Fedus, Jeff Dean, and Barret Zoph. A Review of Sparse Expert Models in Deep Learning, September 2022a. URL <http://arxiv.org/abs/2209.01667>. arXiv:2209.01667 [cs].
- William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity, June 2022b. URL <http://arxiv.org/abs/2101.03961>. arXiv:2101.03961 [cs].
- Yongxin Guo, Zhenglin Cheng, Xiaoying Tang, Zhaopeng Tu, and Tao Lin. Dynamic Mixture of Experts: An Auto-Tuning Approach for Efficient Transformer Models, March 2025. URL <http://arxiv.org/abs/2405.14297>. arXiv:2405.14297 [cs].
- David Ha, Andrew Dai, and Quoc V. Le. HyperNetworks, December 2016. URL <http://arxiv.org/abs/1609.09106>. arXiv:1609.09106 [cs].
- Joelle Hanna, Linus Scheibenreif, and Damian Borth. MAPEX: Modality-Aware Pruning of Experts for Remote Sensing Foundation Models, July 2025. URL <http://arxiv.org/abs/2507.07527>. arXiv:2507.07527 [cs].
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. pp. 770–778, 2016. URL https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.
- Yiding Jiang, Dilip Krishnan, Hossein Mobahi, and Samy Bengio. Predicting the Generalization Gap in Deep Networks with Margin Distributions, June 2019. URL <http://arxiv.org/abs/1810.00113>. arXiv:1810.00113 [stat].
- Alex Krizhevsky. Learning Multiple Layers of Features from Tiny Images.
- Konstantin Schürholt, Boris Knyazev, Xavier Giró-i Nieto, and Damian Borth. Hyper-Representations as Generative Models: Sampling Unseen Neural Network Weights, September 2022a. URL <http://arxiv.org/abs/2209.14733>. arXiv:2209.14733 [cs].
- Konstantin Schürholt, Diyar Taskiran, Boris Knyazev, Xavier Giró-i Nieto, and Damian Borth. Model Zoos: A Dataset of Diverse Populations of Neural Network Models, September 2022b. URL <http://arxiv.org/abs/2209.14764>. arXiv:2209.14764 [cs].
- Konstantin Schürholt, Michael W. Mahoney, and Damian Borth. Towards Scalable and Versatile Weight Space Learning, June 2024. URL <http://arxiv.org/abs/2406.09997>. arXiv:2406.09997 [cs].
- Noam Shazeer. GLU Variants Improve Transformer, February 2020. URL <http://arxiv.org/abs/2002.05202>. arXiv:2002.05202 [cs].
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer, January 2017. URL <http://arxiv.org/abs/1701.06538>. arXiv:1701.06538 [cs].
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, August 2023. URL <http://arxiv.org/abs/1706.03762>. arXiv:1706.03762 [cs].
- Scott Yak, Javier Gonzalvo, and Hanna Mazzawi. Towards Task and Architecture-Independent Generalization Gap Predictors, June 2019. URL <http://arxiv.org/abs/1906.01550>. arXiv:1906.01550 [stat].
- Wenbo Zhao, Yang Gao, Shahan Ali Memon, Bhiksha Raj, and Rita Singh. Hierarchical Routing Mixture of Experts, March 2019. URL <http://arxiv.org/abs/1903.07756>. arXiv:1903.07756 [cs].
- Andrey Zhmoginov, Mark Sandler, and Max Vladymyrov. HyperTransformer: Model Generation for Supervised and Semi-Supervised Few-Shot Learning, July 2022. URL <http://arxiv.org/abs/2201.04182>. arXiv:2201.04182 [cs].
- Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-Experts with Expert Choice Routing, October 2022. URL <http://arxiv.org/abs/2202.09368>. arXiv:2202.09368 [cs].

APPENDIX

A TOP- k GRID SEARCH RESULTS

Table 5: Validation R^2 for the Top- k grid search on the Multi-Zoo. For reference, the dense baseline achieves a Validation R^2 of 0.8088

Number of experts	$k = 1$	$k = 2$	$k = 3$
2	0.8897	0.8486	—
4	0.8456	0.8925	0.8548
6	0.7736	0.8553	0.8547
8	0.7730	0.8904	0.8178

Table 5 reports the validation results of the grid search and Figure 3 visualizes them. In the Multi-Zoo setting the best configuration uses $N = 4$ experts and $k = 2$; this configuration becomes our reference for subsequent routing experiments. The complete Single-Zoo results are deferred to Section B.

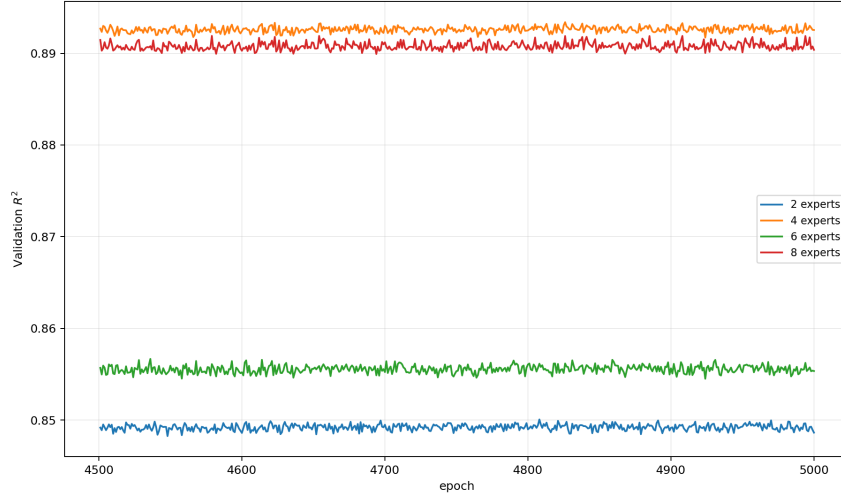


Figure 3: Validation R^2 on the Multi-Zoo for the final 500 epochs. To ensure clarity, only the Top- $k = 2$ configurations are shown, as the performance differences between settings are small. Each line corresponds to a different number of experts N . The best configuration (4 experts) is highlighted in orange.

B SINGLE-ZOO ABLATION RESULTS

To supplement our Multi-Zoo analysis, we report in Table 6 the performance of SANE-MoE on individual datasets (CIFAR-10 and STL-10). These experiments follow the same grid search protocol for the number of experts N and the routing parameter k .

Table 6: Validation R^2 for Single-Zoo ablation studies.

Zoo Config	Experts (N)	Top- $k = 1$	Top- $k = 2$	Top- $k = 3$
CIFAR-10	2	0.8765	0.8816	—
	4	0.8748	0.8888	0.8828
	6	0.8716	0.8924	0.8835
	8	0.8800	0.8972	0.8852
STL-10	2	0.8528	0.8618	—
	4	0.7311	0.8272	0.7632
	6	0.7303	0.7666	0.7383
	8	0.7324	0.7707	0.7421

C ROUTING ANALYSIS

We analyze the routing behaviour of SANE-MoE autoencoder, trained on the **Multi-Zoo** [CIFAR=10 small, STL-10 small]. We use the best Top- k configuration found for this Multi-Zoo setting, i.e. $E = 4$ experts and Top- k gating with $k = 2$. Figure 4 visualizes layer-0 (out of four) of the SANE-MoE encoder routing as a function of token position in the serialized CNN weights sequence. For each zoo, we sample $M = 50$ CNN checkpoints, tokenize each checkpoint into a fixed-length sequence ($T = 15$, enforced by the projection head), run the frozen SANE model, and record, for simplicity, just the *Top-1* expert among the k selected at each position. Panels (a,b) show the empirical $\hat{P}(e \mid p)$ (column-normalized counts over the M checkpoints). Panel (c) shows the difference $\hat{P}_{\text{CIFAR}}(e \mid p) - \hat{P}_{\text{STL}}(e \mid p)$. The near-identical banded structure indicates that layer-0 routing is primarily position-driven and largely dataset-agnostic.

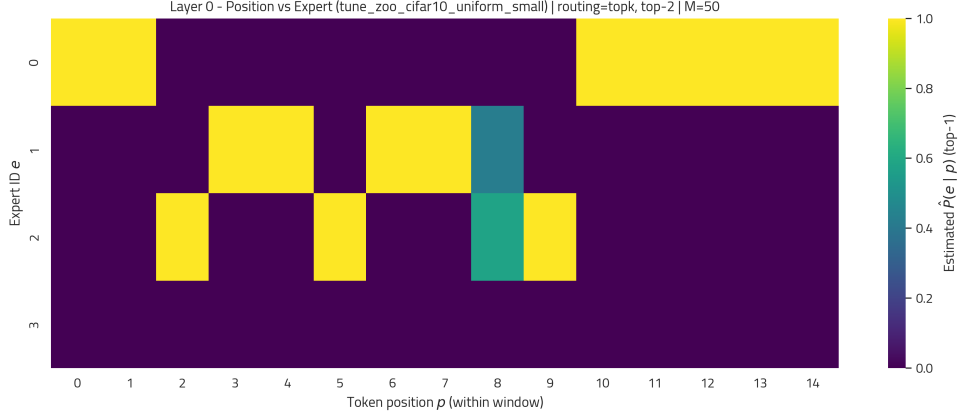
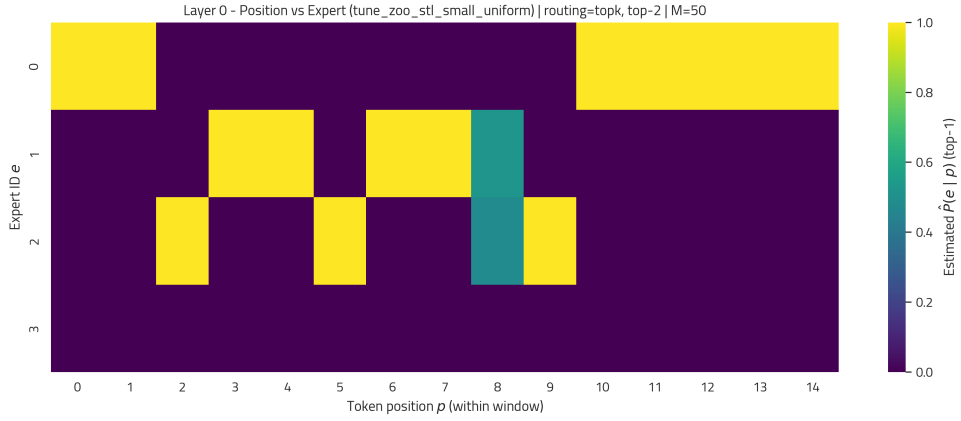
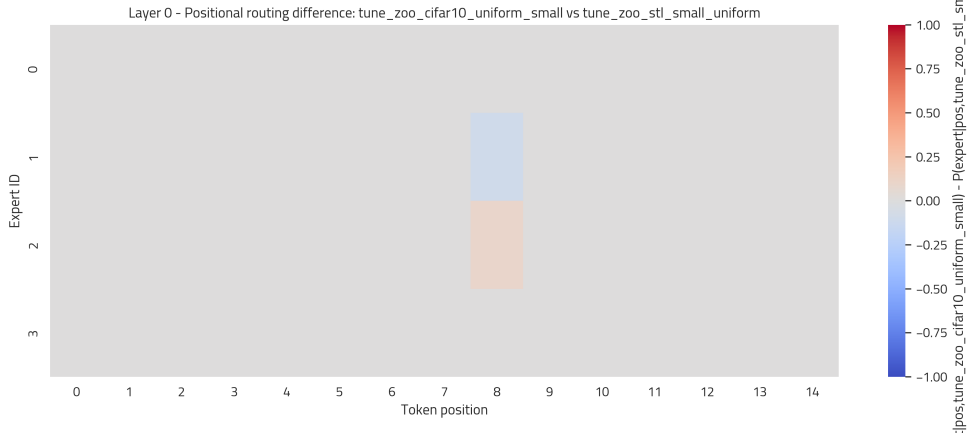
(a) CIFAR-10 zoo: $\hat{P}(e | p)$ (b) STL-10 zoo: $\hat{P}(e | p)$ (c) Difference: $\hat{P}_{\text{CIFAR}}(e | p) - \hat{P}_{\text{STL}}(e | p)$

Figure 4: In **layer-0** of the MoE encoder **Routing** is position-driven and dataset-agnostic. Panels (a,b) show the estimated conditional distribution of Top-1 expert selection given token position, $\hat{P}(e | p)$, computed by aggregating routing decisions over multiple tokenized model checkpoints from each zoo. Panel (c) shows the per-position difference between CIFAR-10 and STL-10.

By layer 2, routing differs between CIFAR-10 and STL-10 in a position-localized manner (??). This divergence yields a moderate expert–dataset association: some experts become preferentially used by one zoo, while others remain mixed, indicating partial (non-exclusive) specialization rather than a strict dataset–expert partition.



Figure 5: **Layer-2 routing differences across zoos.** Heatmap of the difference $\hat{P}_{\text{CIFAR}}(e | p) - \hat{P}_{\text{STL}}(e | p)$, where $\hat{P}(e | p)$ is estimated from Top-1 expert selections (among Top- k , $k = 2$) aggregated over $M = 50$ checkpoints per zoo (fixed $T = 15$ tokens). Positive values (red) indicate higher expert usage for CIFAR-10 at position p , negative values (blue) indicate higher usage for STL-10, and values near zero indicate similar routing.

D DETAILED DESCRIPTION OF THE DOWNSTREAM TASK

The ”Downstream Generative Task” is performed on following what is described in Schürholt et al. (2024). In order to assess the quality of the learned *hyper-representations* and the generative capabilities of our SANE-MoE model, we designed the downstream task that operates in two distinct modes: **accuracy of Reconstructed models** and **accuracy of novel generated models**.

Accuracy of Reconstruction(acc_{Rec}) This mode evaluates the capability of the autoencoder to compress and recover neural network weights. Given a validation set of trained model checkpoints, $\mathcal{W}_{\text{val}}$, we encode each model $\theta \in \mathcal{W}_{\text{val}}$ and then immediately decode it back to obtain $\hat{\theta}$:

$$\hat{\theta} = \text{Decoder}(\text{Encoder}(\theta))$$

The quality of the reconstruction is not measured parameter-wise Mean Squared Error (MSE), but by the functional accuracy retention. This means that we evaluate the reconstructed model $\hat{\theta}$ on its original task (e.g., CIFAR-10 (actual) test set for classification) and compare it to the accuracy of the **original** model θ , this is what we call **Accuracy of Reconstruction** acc_{Rec} . We then compute the accuracy difference $\Delta_{\text{acc}_{\text{Rec}}}$:

$$\Delta_{\text{acc}_{\text{Rec}}} = \text{acc}_{\text{Rec}\hat{\theta}} - \text{acc}_{\text{Rec}\theta},$$

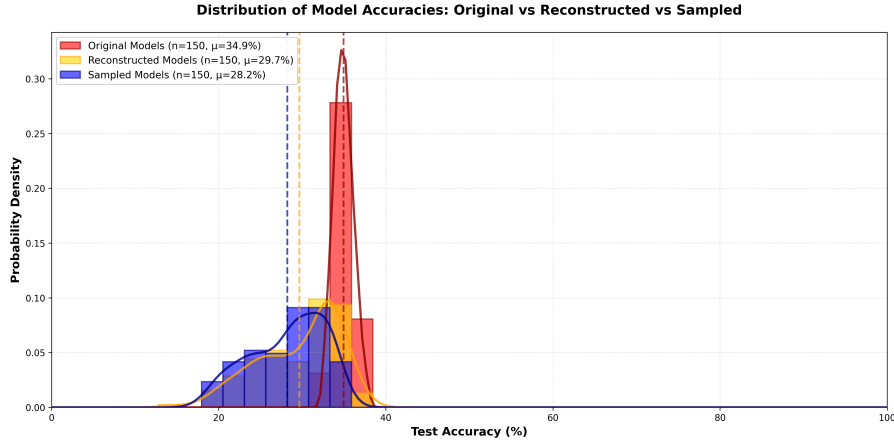
which quantifies how well the SANE encoder preserves the essential functional properties of the original weights. In details, we sample $N = 5$ models per architecture from the validation set, these are encoded, decoded, then we perform classification on the original dataset and report the Reconstruction Accuracy Mean and $\Delta_{\text{acc}_{\text{Rec}}}$ average accuracy drop.

Generating new models via latent sampling To test if the learned latent space Z allows for the generation of valid, unseen models, we employ a sampling strategy based on Kernel Density Estimation (KDE). We select a subset of ”anchor” models from the validation set, specifically the top-100 performing models. This ensures we model the distribution of high-quality solutions. Then, we compute the latent embeddings of these anchors. To capture the complex geometry of the weight manifold, we fit a Per-Token KDE sampler with a Gaussian kernel and a specific bandwidth (e.g.,

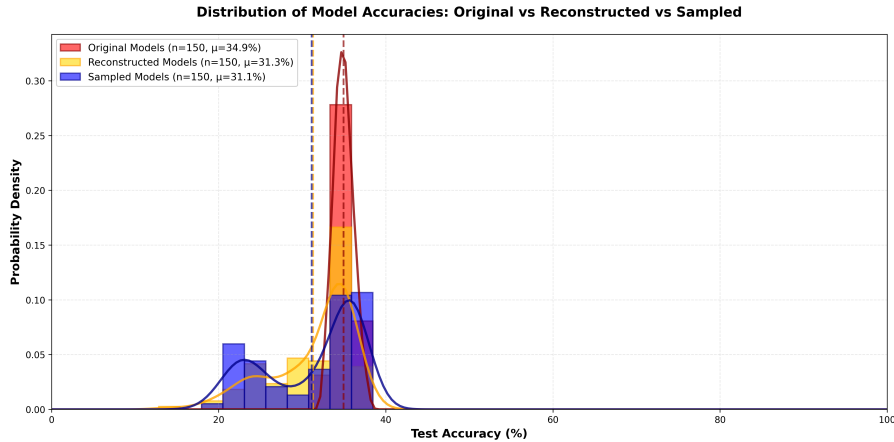
$h = 0.05$) to these embeddings. After that, we sample 50 new latent codes z_{new} from the fitted KDE distribution and decode them into weight space. Finally, since generated weights may have slightly shifted activation statistics, we perform a single pass of Batch Normalization conditioning (updating running mean/variance) using a subset of target data before evaluation. Once we have these 50 newly generated models, we compute the test set accuracy on the classification task.

E DOWNSTREAM TASK RESULTS

We report here for completeness also the plot (Figure 6) of the probability density of test accuracies over approximately 150 models on **STL-10 small Zoo** for a visual comparison between SANE-standard and SANE-MoE. As for CIFAR-10 small Zoo also for STL-10 small Zoo SANE-MoE reconstructed and generated models are closer to the original CNNs accuracies than the one produced by SANE-standard. It is also worth highlighting that in this case, for the generated models, the standard deviation is higher, and a discrete number of generated models are on average even better than the original one.



(a) SANE-standard



(b) SANE-MoE (position-aware)

Figure 6: Probability density of test accuracy over approximately 150 models on **STL-10 small Zoo**. Original CNN models are shown in red, reconstructed models in yellow, and sampled models in blue. (a) Results obtained using SANE-standard for reconstruction and generation, trained on Multi-Zoo (CIFAR-10 + STL-10). (b) Results obtained using SANE-MoE with position-aware routing, trained on Multi-Zoo (CIFAR-10 + STL-10).

F TRAINING HYPERPARAMETERS

Both SANE-standard and all SANE-MoE variants are trained using an identical optimization and training protocol. This includes the same optimizer, learning-rate schedule, batch size, number of training epochs, and regularization settings. A complete overview of the shared training hyperparameters is provided in Table 7.

Table 7: Training hyperparameters shared across SANE-standard and SANE-MoE models.

Category	Value
Trainer	SANEContrastiveTrainer
Random seed	2020
Device	CUDA
Precision	Automatic mixed precision (bfloat16)
Epochs	5000
Batch size	128
Validation frequency	Every epoch
Checkpoint frequency	Every 250 epochs
Gradient clipping	None
Mask probability	0.0
Additive noise	0.0
Multiplicative noise	0.0
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	3×10^{-9}
Scheduler	OneCycleLR
Input dimension	230
Latent dimension	128
Transformer layers	4
Attention heads	4
Model width (d_{model})	512
Projection dimension	30
Loss function	Composite loss
Contrastive temperature	0.1
Reconstruction weight (γ)	0.05
MoE balance loss weight	0.01
GPUs	1
CPUs	32
Data loader workers	4